

RefriSmart-AI Interview Study Guide

RefriSmart-AI — Complete Interview Study Guide

1. HOW THE WEB WORKS (Foundation)

Q: What happens when you open a website? - You type a URL → browser asks DNS for IP address → browser sends HTTP request to server → server sends back HTML/CSS/JS → browser renders it.

HTTP Methods: | Method | Use | |—|—| | GET | Fetch data (no body) | | POST | Send new data | | PUT/PATCH | Update existing data | | DELETE | Remove data |

HTTP Status Codes: | Code | Meaning | |—|—| | 200 | OK | | 201 | Created | | 400 | Bad Request (user error) | | 401 | Unauthorized (not logged in) | | 403 | Forbidden (not allowed) | | 404 | Not Found | | 500 | Server Error |

What is an API? An API (Application Programming Interface) is a way for two systems to talk to each other. In this project, the frontend (Next.js) calls the backend (Express) using REST API endpoints like `/api/auth/login`.

What is REST? REST = Representational State Transfer. Rules for designing APIs: - Use HTTP methods correctly (GET, POST, etc.) - Each URL represents a resource (`/users`, `/products`) - Stateless — each request is independent

2. FRONTEND — React & Next.js

What is React? React is a JavaScript library for building UIs using components. Instead of writing HTML directly, you write reusable pieces (components) that React assembles into a page.

What is a Component? A function that returns HTML (JSX). Example:

```
function Button({ text }) {
  return <button>{text}</button>;
}
```

What is State? (useState) State = data that changes over time. When state changes, React re-renders the component.

```
const [count, setCount] = useState(0);
// count is the value, setCount changes it
```

What is useEffect? Runs code after the component renders. Used for API calls, timers, etc.

```
useEffect(() => {
  fetch('/api/products').then(...)
}, []); // [] means run once on load
```

What is a Hook? Functions starting with use that add features to components. Common hooks: useState, useEffect, useContext, useRef.

What is Context? (useContext) A way to share data (like user login info) across many components without passing props through every level.

What is Next.js? Next.js is a framework built on top of React that adds: - **File-based routing** — create app/about/page.tsx → automatically creates /about page - **Server-Side Rendering (SSR)** — page HTML generated on server (good for SEO) - **Static Generation (SSG)** — pages built at deploy time (very fast) - **API Routes** — write backend code inside Next.js

What is the App Router? (Next.js 13+) New routing system where each folder in app/ is a route. page.tsx = the page, layout.tsx = shared wrapper, loading.tsx = loading state.

Difference: SSR vs CSR vs SSG | Type | When HTML is made | Good for | |
 —|—|—| | SSR | On every request (server) | Dynamic pages, SEO | | CSR | In browser (client) | Dashboards, no SEO needed | | SSG | At build time | Blogs,

static content |

What is TypeScript? JavaScript with types. You tell the compiler what type each variable is — prevents bugs at compile time before running.

```
// JavaScript
```

```
function add(a, b) { return a + b; }
```

```
// TypeScript
```

```
function add(a: number, b: number): number { return a + b; }
```

What is an Interface? A TypeScript contract that defines the shape of an object:

```
interface User {  
  id: string;  
  name: string;  
  email: string;  
}
```

What is Tailwind CSS? A CSS framework with utility classes. Instead of writing CSS files, you add classes directly to HTML:

```
<button class="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-700">
```

Click Me

```
</button>
```

3. BACKEND — Node.js & Express

What is Node.js? Node.js lets you run JavaScript on the server (outside the browser). It's event-driven and non-blocking — great for handling many requests simultaneously.

What is Express.js? A minimal web framework for Node.js. Makes it easy to create API routes:

```
const app = express();  
app.get('/api/products', (req, res) => {  
  res.json({ products: [...] });  
});  
app.listen(5001);
```

What is Middleware? Functions that run between the request and response. Like checkpoints:

Request → [CORS Middleware] → [Auth Middleware] → [Controller] → Response

Examples in this project: - `cors()` — allows frontend to call backend - `cookieParser()` — reads cookies from requests - `authMiddleware` — checks if user is logged in

What is a Controller? A function that handles a specific API route. In this project: - `authController.ts` — login, register, OTP - `aiController.ts` — Gemini AI calls - `adminController.ts` — admin CRUD operations - `productController.ts` — products and orders

What is CORS? Cross-Origin Resource Sharing. By default, browsers block requests to a different domain. CORS headers tell the browser “it’s okay, this domain is allowed.”

In this project: Frontend (localhost:3000) calls Backend (localhost:5001) — CORS is needed.

What are Environment Variables (.env)? Secret values stored outside the code:

```
JWT_SECRET="mysecret"
GEMINI_API_KEY="AIza..."
```

Never commit `.env` to GitHub. Access in code via `process.env.JWT_SECRET`.

4. AUTHENTICATION — JWT & OTP

What is Authentication vs Authorization? - **Authentication** = Who are you? (Login) - **Authorization** = What can you do? (Admin vs User)

What is JWT (JSON Web Token)? A token that proves who you are. Has 3 parts separated by dots:

`header.payload.signature`

- **Header** — token type, algorithm
- **Payload** — user data (id, role, email)
- **Signature** — proof the token wasn’t tampered with

How JWT works in this project: 1. User logs in → server creates JWT with user ID 2. JWT sent to browser as HTTP-only cookie 3. Every future request sends the cookie automatically 4. Server reads cookie, verifies JWT, knows who the user is

What is HTTP-only Cookie? A cookie JavaScript cannot access (prevents XSS attacks). Only the browser sends it automatically with requests.

What is bcryptjs? Library for hashing passwords. Never store passwords as plain text:

```
// Saving password
const hashed = await bcrypt.hash(password, 10);

// Checking password
const isMatch = await bcrypt.compare(enteredPassword, hashed);
```

What is a Hash? A one-way transformation — you can convert password → hash but NOT hash → password.

What is OTP? One-Time Password. In this project, when you register: 1. Server generates a 6-digit random code 2. Sends it to your email via Nodemailer 3. User enters the code → server verifies it 4. Account is marked as verified

5. DATABASE — PostgreSQL & Prisma

What is a Database? Persistent storage for data. Data survives even if the server restarts.

What is PostgreSQL? A powerful open-source relational database. Data stored in tables with rows and columns (like Excel sheets that are linked together).

What is SQL? Structured Query Language — language used to talk to relational databases:

```
SELECT * FROM users WHERE email = 'test@gmail.com';
INSERT INTO products (name, price) VALUES ('AC Filter', 299);
UPDATE orders SET status = 'completed' WHERE id = '123';
```

What is an ORM? Object Relational Mapper — lets you write database queries using your programming language instead of SQL:

// Instead of raw SQL, use Prisma:

```
const user = await prisma.user.findUnique({ where: { email } });
```

What is Prisma? The ORM used in this project. You define your database structure in `schema.prisma`:

```
model User {
  id          String    @id @default(uuid())
  name        String
  email       String    @unique
  password    String
  role        Role      @default(USER)
  createdAt   DateTime  @default(now())
  services    Service[]
}
```

What are Relations? Links between tables: - One User can have MANY Services (one-to-many) - One Order belongs to ONE Product

What is a Migration? When you change your schema, a migration updates the actual database to match. `prisma migrate dev` creates a migration file and applies it.

What is UUID? Universally Unique Identifier — a random string used as a primary key instead of 1, 2, 3... More secure and works in distributed systems.

What is Neon/Supabase? Managed PostgreSQL hosting services — like Google Drive but for databases. No need to manage your own server.

6. AI INTEGRATION — Google Gemini

What is an AI API? An API provided by a company (Google) that lets you send data and get AI-generated responses back.

What is Google Gemini? Google's AI model (like ChatGPT). Gemini Vision can analyze images and videos.

How AI Diagnosis works in this project: 1. User uploads photo/video of broken appliance 2. Multer saves file temporarily on the server 3. File is uploaded to Cloudinary (gets a URL) 4. Cloudinary URL + description sent to Gemini API 5. Gemini returns: fault type, severity, repair steps, cost estimate 6. Response saved in database + shown to user

What is a Prompt? The instruction you give to an AI. In this project, the prompt tells Gemini: “You are an appliance repair expert. Analyze this image and identify the fault...”

What is Multer? Express middleware for handling file uploads (multipart/form-data). It processes the uploaded file from the request before the controller runs.

7. CLOUD STORAGE — Cloudinary

Why not store files on the server? - Serverless functions are temporary (no disk) - Files would be deleted on redeploy - No global CDN = slow for distant users

What is Cloudinary? A cloud service for storing and serving images and videos. Features: - Global CDN — serves files from nearest server worldwide - Auto image optimization - Transformation via URL (resize, crop, compress)

How it works in this project: 1. User uploads file → Multer gets it as buffer 2. Buffer uploaded to Cloudinary via API 3. Cloudinary returns a secure URL 4. URL stored in PostgreSQL database 5. Frontend displays the image using that URL

8. PAYMENTS — Razorpay

What is a Payment Gateway? A service that securely processes online payments. Like a middleman between customer and bank.

How Razorpay works in this project: 1. Customer clicks “Book Service” (₹349 visiting charge) 2. Backend calls Razorpay API → creates an Order with an Order ID 3. Order ID sent to frontend 4. Frontend opens Razorpay payment popup 5. Customer pays → Razorpay sends success callback 6. Frontend sends payment details to backend 7. Backend verifies payment signature (security check) 8. If verified → service booking confirmed in database

Why verify the signature? To prevent fake payment confirmations. Razorpay signs the payment with a secret key — you verify it’s real.

9. EMAIL — Nodemailer

What is Nodemailer? A Node.js library for sending emails from your server.

SMTP: Simple Mail Transfer Protocol — the protocol for sending emails. Gmail provides SMTP access with an App Password.

How OTP email works:

```
const transporter = nodemailer.createTransporter({
  host: 'smtp.gmail.com',
  port: 587,
  auth: { user: 'email@gmail.com', pass: 'app_password' }
});
await transporter.sendMail({
  to: user.email,
  subject: 'Your OTP Code',
  html: `

Your OTP is: <b>123456</b></p>`
});


```

10. DEPLOYMENT — Vercel & Serverless

What is Deployment? Making your app available on the internet, not just on your computer.

What is Vercel? A cloud platform specializing in Next.js and serverless deployments. Free tier available.

What is a Serverless Function? Code that runs on demand — no permanent server. The cloud provider spins up a container, runs your function, and shuts it down. You pay per request, not per hour.

In this project: - Frontend → Vercel CDN (globally distributed) - Backend → Vercel Serverless Functions (Express as one function) - Database → Neon (serverless PostgreSQL)

What is CI/CD? Continuous Integration / Continuous Deployment: - Push code to GitHub → Vercel automatically builds and deploys - No manual deployment steps needed

What is a CDN? Content Delivery Network — copies of your files stored worldwide. Users get files from the nearest server = faster loading.

11. SEO (Search Engine Optimization)

What is SEO? Making your website rank higher on Google search results.

What is JSON-LD Schema? Structured data that tells Google exactly what your page is about:

```
<script type="application/ld+json">
{
  "@type": "LocalBusiness",
  "name": "Golden Refrigeration",
  "address": { "addressLocality": "Bhagalpur" }
}
</script>
```

What is Open Graph? Meta tags that control how your page looks when shared on WhatsApp, Twitter, Facebook:

```
<meta property="og:title" content="Golden Refrigeration" />
<meta property="og:image" content="/banner.jpg" />
```

What is a Sitemap? An XML file listing all pages of your website. Helps Google discover and index all pages.

What is Robots.txt? A file that tells search engine crawlers which pages to index and which to skip.

12. PROJECT ARCHITECTURE EXPLAINED

What is a Monorepo? Both frontend and backend code in ONE git repository. Easier to manage for small teams.

Frontend → Backend Communication:

User clicks "Book Service"

→ Frontend sends POST request to /api/service/book

→ Request includes JWT cookie (auto-sent by browser)

- Backend auth middleware verifies JWT
- Controller creates service record in PostgreSQL via Prisma
- Returns { success: true, serviceId: "..." }
- Frontend shows success toast notification

What is the MVC Pattern? (Model-View-Controller) - Model = Database (Prisma schema) - **View** = Frontend (Next.js pages) - **Controller** = Backend controllers (authController, aiController...)

13. COMMON INTERVIEW QUESTIONS & ANSWERS

Q: Why did you choose Next.js over plain React? Next.js gives us SSR for SEO (important for local search ranking), file-based routing (simpler code structure), and built-in image optimization. For a service business, appearing on Google search is critical.

Q: Why PostgreSQL over MongoDB? Our data is highly relational — Users have Services, Services have Payments, Users have Diagnoses. PostgreSQL enforces these relationships at the database level, preventing orphaned records. MongoDB is better for unstructured data.

Q: How do you protect your API routes? JWT middleware runs before every protected route. It reads the cookie, verifies the JWT signature using the secret key, and either allows the request to continue or returns 401 Unauthorized.

Q: What happens if the Gemini API is down? The service booking still works. AI diagnosis is a separate feature — if Gemini fails, we catch the error and return a user-friendly message. The core booking flow is not dependent on AI.

Q: How do you handle file uploads securely? Multer validates file type and size before processing. Files go straight to Cloudinary — never stored permanently on the server. The server only holds the file in memory briefly during the upload.

Q: What is the difference between authentication and authorization? Authentication = verifying identity (login). Authorization = verifying permissions (admin vs regular user). In this project, after JWT authentication, we check the user's role field from the database to authorize admin routes.

Q: What are the security measures in the project? 1. Passwords hashed with bcrypt (never stored plain) 2. JWT in HTTP-only cookies (prevents XSS theft) 3. Email OTP verification (prevents fake accounts) 4. Razorpay signature verification (prevents fake payments) 5. Environment variables for all secrets (not in code) 6. Role-based access control for admin routes

Q: How does the Razorpay payment flow work? 1. Backend creates a Razorpay order (gets order_id) 2. Frontend opens checkout with order_id 3. Customer pays → Razorpay callback fires 4. Frontend sends payment_id + order_id + signature to backend 5. Backend uses HMAC SHA256 to verify the signature 6. Only if verified, the booking is confirmed

Q: What is the difference between cookies and localStorage? | Feature | Cookies | localStorage | |—|—|—| | Sent with requests | Yes (automatic) | No | | Accessible by JS | Depends on HttpOnly | Yes | | Expiry | Configurable | Never expires | | Size limit | 4KB | 5MB | We use HTTP-only cookies for JWT so JavaScript cannot steal it.

Q: What is Prisma's role in the project? Prisma is our ORM. We define the schema once in `schema.prisma` and Prisma generates a type-safe TypeScript client. Every database query is type-safe — TypeScript catches errors at compile time, not runtime.

Q: How does your app handle the case when a user is not verified? The `authMiddleware` checks the `isVerified` field on the user. If false, it returns 403 Forbidden. Users must complete OTP verification before accessing protected features.

Q: What is serverless and why use it? Serverless means no permanent server process. The backend runs as functions that spin up on request. Benefits: auto-scaling (handles traffic spikes), pay-per-use (cheaper), zero server maintenance. Trade-off: cold starts can add ~200ms on first request.

Q: How do you manage state in Next.js? Combination of approaches: - **useState** for local component state (form inputs, toggles) - **useContext** for global state (logged-in user info) - **URL params** for navigation state (page filters) - **Server fetching** for data that comes from the API

Q: What is the difference between prisma migrate dev and prisma db push? - `db push` — fast, syncs schema to DB directly, no migration history. Good for prototyping. - `migrate dev` — creates a migration file with the changes. Maintains full history. Needed for production.

Q: Explain the folder structure of your Next.js app. The app/ directory uses Next.js App Router. Each subfolder is a route — app/login/page.tsx creates the /login page. layout.tsx wraps all pages with shared navbar/footer. _dashboard.tsx files with underscore prefix are not routes — they're private component files.

14. TECHNOLOGY COMPARISONS TO KNOW

Question	Answer
React vs Next.js	React is a library; Next.js is a framework built on React that adds routing, SSR, etc.
SQL vs NoSQL	SQL: structured, relational; NoSQL: flexible, document-based
REST vs GraphQL	REST: multiple endpoints for each resource; GraphQL: one endpoint, client specifies exact data needed
npm vs npx	npm installs packages; npx runs packages without installing
async/await vs callbacks	async/await is modern, readable; callbacks lead to “callback hell”
JWT vs Sessions	JWT: stateless, stored client-side; Sessions: stateful, stored server-side

15. KEY TERMS GLOSSARY

Term	Simple Meaning
API	A way for two systems to talk
Endpoint	A specific URL in an API (e.g., /api/login)
Request	Message sent from browser to server
Response	Message sent back from server to browser
JSON	Data format using key-value pairs (like a dictionary)
Async	Code that doesn't block — runs in background

Term	Simple Meaning
Promise	An object that will have a value in the future
Middleware	Code that runs between request and response
ORM	Tool to use database with your coding language
CDN	Network of servers worldwide for fast file delivery
Serverless	Code runs on demand, no permanent server
Build	Compiling/optimizing code for production
Deploy	Making app live on the internet
Repository (Repo)	A project folder tracked by git
Commit	A saved snapshot of code changes
Branch	A separate line of development
Environment Variable	Secret config values stored outside code
Hash	One-way transformation (password security)
Token	A string that proves identity or access
Schema	The structure/blueprint of your database
Migration	Updating database structure
Payload	The data inside a request or JWT
CORS	Permission for one domain to call another
SSL/HTTPS	Encrypted connection between browser and server